

Projeto — Otimização Computacional em IA

Enquadramento do projeto

Seguindo uma abordagem *Problem-Based Learning*, a avaliação da UC de Otimização Computacional inclui como elemento principal o desenvolvimento de um projeto aplicado centrado na formulação e resolução de um **problema de otimização no contexto aplicativo da IA**. O projeto:

- **avalia a capacidade de pensar em termos de otimização aplicada à IA e não apenas de aplicar algoritmos**, tendo por base os objetivos abaixo indicados;
- deve evidenciar a integração de conhecimentos teóricos com competências práticas em modelação, implementação e análise crítica;
- é desenvolvido em trabalho de grupo, com **2 estudantes cada por grupo**, em que ambos os estudantes devem ter **participação ativa em todas as fases/tutorias**;
- tem **nota mínima de 9,5 valores** (escala 0-20) e contribui para 80% da classificação final na UC (nota: os restantes 20%(=2x10%) são relativos aos 2 quizzes *online* individuais).

Objetivos do projeto

- Formular um problema de otimização no contexto da IA
- Selecionar e implementar métodos de otimização
- Analisar convergência, estabilidade e desempenho
- Comparar abordagens e justificar escolhas
- Propor melhorias devidamente fundamentadas

Tema do projeto

Cada grupo de 2 estudantes deverá selecionar uma aplicação a partir da lista abaixo ou, em alternativa, propor uma aplicação sujeita a aprovação.

Fases do projeto e datas importantes

- Fase 1 (1ª tutoria) (20% da classificação da UC): plano de trabalho (3-4 páginas) com identificação e formulação do problema de otimização e métodos/técnicas de otimização a usar.
Deadline de submissão: **18 de maio** (até às 23:59)
- Fase 2 (2ª tutorial) (60% da classificação da UC): projeto final com implementação dos métodos, resultados experimentais, análise crítica do desempenho, comparação de métodos aplicados e propostas e melhoria. Esta Fase 2 termina com uma defesa do trabalho desenvolvido.

Deadline de submissão: **11 de junho** (até às 23:59)

Defesa online: data a definir no período **de 16 a 18 de junho**

Formatos de entrega do projeto

Opção A: Jupyter Notebook com texto, código e resultados.

Opção B: Relatório (15-20 páginas) com código em anexo.

Lista de propostas de aplicação em IA

Aplicações que já estavam nos slides da Aula 1

- **Treino de modelos supervisionados** (*supervised learning training*)
Minimizar uma função de perda: erro quadrático, entropia cruzada, hinge loss. É a formulação base de regressão, classificação e redes neurais.
- **Regressão linear e mínimos quadrados** (*linear regression and least squares*)
Resolver um problema de mínimos quadrados, muitas vezes convexo, para ajustar coeficientes.
- **Máquinas de vetores de suporte** (*SVM-support vector machines*)
Maximizar a margem entre classes, geralmente através de programação quadrática convexa com restrições.
- **Publicidade computacional e leilões algorítmicos** (*computational advertising and ad auctions*)
Maximizar receita, relevância ou utilidade sob restrições de orçamento, fairness e ranking.
- **Regressão logística** (*logistic regression*)
Minimizar perda log-loss/entropia cruzada, geralmente com métodos de gradiente.
- **Planeamento sequencial** (*sequential decision-making/planning*)
Resolver um problema de decisão em múltiplos passos, onde cada ação altera o estado e o processo de otimização procura a sequência ótima.
- **Planeamento de trajetórias em robótica** (*robot trajectory optimisation*)

Encontrar trajetórias que minimizam energia, tempo, risco de colisão ou erro, sob restrições físicas e cinemáticas.

- **Visão computacional** (*computer vision*)
Segmentação, reconstrução, estimação de pose, calibração e matching podem ser formulados como minimização de erro geométrico/fotométrico.

Aplicações que não estavam nos slides da Aula 1

- **Regularização L1, L2 e Elastic Net** (*regularisation*)
Acrescentar penalizações à função objetivo para controlar sobreajustamento; L1 induz esparsidade e L2 penaliza pesos grandes.
- **Seleção de variáveis/características** (*feature selection*)
Escolher subconjuntos de variáveis relevantes; pode ser vista como otimização combinatória ou como penalização esparsa.
- **Redução de dimensionalidade** (*dimensionality reduction*)
PCA, autoencoders e métodos afins minimizam erro de reconstrução ou maximizam variância explicada.
- **Agrupamento de dados** (*clustering*)
K-means minimiza a soma das distâncias quadráticas aos centróides; outros métodos usam objetivos de separação/coerência.
- **Treino de redes neurais profundas** (*deep neural network training*)
Problema não convexo, de grande dimensão, resolvido por SGD, Adam e variantes de gradiente estocástico.
- **Descida do gradiente estocástica e variantes** (*SGD, Adam, adaptive optimisation*)
Otimização iterativa usando amostras ou mini-batches para escalar o treino a grandes bases de dados.
- **Afinação de hiperparâmetros** (*hyperparameter optimisation*)
Procurar valores ótimos para learning rate, profundidade, regularização, número de árvores, etc. Usa grid search, random search, otimização Bayesiana.

- **Otimização Bayesiana** (*Bayesian optimisation*)
Otimização global de funções caras de avaliar, usando modelos substitutos e funções de aquisição. Muito usada em AutoML e afinação de modelos.
- **AutoML e seleção automática de pipelines** (*automated machine learning*)
Escolher automaticamente pré-processamento, modelo, arquitetura e hiperparâmetros. É otimização sobre espaços mistos: contínuos, discretos e combinatórios.
- **Pesquisa de arquiteturas neuronais** (*neural architecture search, NAS*)
Otimizar a arquitetura de uma rede: número de camadas, ligações, blocos, operações. Usa RL, evolução, gradiente e otimização Bayesiana.
- **Compressão de modelos** (*model compression*)
Reduzir custo computacional mantendo desempenho: problema de compromisso entre precisão, memória, latência e energia.
- **Poda de redes neuronais** (*neural network pruning*)
Remover pesos, neurónios ou camadas pouco relevantes, minimizando perda de desempenho sob restrições de tamanho ou latência.
- **Quantização de redes neuronais** (*neural network quantisation*)
Aproximar pesos e ativações por valores de menor precisão; otimiza memória, velocidade e energia, sobretudo em IA embebida/edge AI.
- **Destilação de conhecimento** (*knowledge distillation*)
Treinar um modelo pequeno para aproximar as saídas de um modelo grande, minimizando uma perda entre distribuições preditivas.
- **Aprendizagem por reforço** (*reinforcement learning*)
Maximizar recompensa esperada acumulada; inclui equações de Bellman, programação dinâmica, policy gradient e controlo ótimo.
- **Otimização de políticas** (*policy optimisation*)
Ajustar diretamente os parâmetros de uma política para maximizar retorno esperado; base de métodos como REINFORCE, actor-critic e policy gradient.
- **Controlo preditivo baseado em modelo** (*model predictive control, MPC*)

Resolver repetidamente um problema de otimização de trajetória, executar a primeira ação e re-otimizar no passo seguinte.

- **SLAM e visão robótica** (*SLAM and bundle adjustment*)
Estimar simultaneamente posição da câmara/robô e estrutura do ambiente; tipicamente mínimos quadrados não lineares.
- **Sistemas de recomendação** (*recommender systems*)
Fatorização matricial minimiza erro entre avaliações observadas e previstas; pode incluir regularização e vieses de utilizador/item.
- **Aprendizagem para ordenação** (*learning to rank*)
Otimizar rankings em pesquisa, recomendação ou publicidade; idealmente otimizar métricas como NDCG, MAP ou aproximações diferenciáveis.
- **Embeddings de palavras** (*word embeddings*)
Aprender vetores que minimizam perdas de previsão contextual, como negative sampling ou NCE.
- **Modelos de linguagem** (*language models*)
Treino por minimização de entropia cruzada/negative log-likelihood; inferência por procura aproximada de sequências prováveis.
- **Descodificação em NLP** (*decoding, beam search*)
Procurar a sequência de tokens com maior probabilidade aproximada; beam search é uma heurística de otimização discreta.
- **Tradução automática neural** (*neural machine translation*)
O treino minimiza perda sequencial; a geração usa descodificação aproximada, como greedy search ou beam search.
- **IA generativa adversarial** (*generative adversarial networks, GANs*)
Problema minimax entre gerador e discriminador; formalmente um jogo de dois jogadores com ponto de sela.
- **Modelos de difusão** (*diffusion models*)

Otimizam objetivos relacionados com denoising, score matching, maximum likelihood e ELBO.

- **Autoencoders variacionais** (*variational autoencoders, VAEs*)
Maximizar uma cota inferior variacional, ELBO; envolve inferência aproximada e otimização estocástica.
- **Ataques adversariais** (*adversarial attacks*)
Encontrar pequenas perturbações que maximizam o erro do modelo; é otimização restrita em torno dos dados originais.
- **Treino adversarial e robustez** (*adversarial training and robustness*)
Problema min-max: minimizar a perda no pior caso dentro de uma bola de perturbações admissíveis.
- **Explicações contrafactuais** (*counterfactual explanations*)
Procurar a menor alteração nos atributos de entrada que mude a decisão do modelo. É um problema de otimização com restrições plausíveis.
- **IA explicável baseada em otimização** (*optimisation-based explainable AI*)
Encontrar explicações esparsas, regras locais, exemplos representativos ou alterações mínimas que justifiquem uma previsão.
- **Fairness em aprendizagem automática** (*fairness-constrained machine learning*)
Treinar modelos com restrições de equidade: paridade estatística, igualdade de oportunidades, limites por grupo.
- **Compromisso precisão–equidade** (*accuracy–fairness trade-off*)
Problema multiobjetivo: maximizar desempenho preditivo e, simultaneamente, reduzir enviesamento.
- **Privacidade diferencial no treino** (*differentially private optimisation*)
Adaptar SGD/SGDA com ruído e clipping para limitar fuga de informação individual.
- **Aprendizagem federada** (*federated learning*)
Otimização distribuída: vários clientes atualizam modelos localmente e agregam gradientes/parâmetros sem centralizar dados.

- **Otimização multiobjectivo em IA** (*multi-objective optimisation in AI*)
Equilibrar vários critérios: precisão, custo, interpretabilidade, energia, fairness, latência. Usa frentes de Pareto.
- **IA em edge computing e dispositivos embebidos** (*edge AI optimisation*)
Otimizar modelos para correrem em dispositivos com pouca memória, energia e capacidade computacional.
- **Otimização de inferência** (*inference optimisation*)
Reduzir latência e custo de execução: batching, quantização, poda, compilação de grafos e escolha de hardware.
- **Aprendizagem semi-supervisionada e auto-supervisionada** (*semi/self-supervised learning*)
Construir objetivos auxiliares: reconstrução, contraste, previsão de partes ocultas, alinhamento de representações.
- **Aprendizagem contrastiva** (*contrastive learning*)
Otimizar uma função que aproxima exemplos semelhantes e afasta exemplos diferentes no espaço latente.
- **Modelos probabilísticos e inferência variacional** (*probabilistic models and variational inference*)
Transformar inferência probabilística difícil num problema de otimização sobre distribuições aproximadas.
- **Programação diferenciável** (*differentiable programming*)
Incorporar módulos de otimização, simulação ou decisão dentro de redes treináveis por gradiente.